



Slides for P3740R0

Last chance to fix `std::nontype`

Document number:

P3753R0

Date:

2025-06-17

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/nontype-slides.cow



IMPORTANT: This document has custom controls:

- , : go to the next slide
- , : go to previous slide

Last chance to fix `std::nontype` P3740R0

Introduction

- P2472R3 added `std::nontype` constructors for `std::function_ref`
- merged into P0792R14, plenary-approved Varna 2023 for C++26
- `std::nontype` possibly obsoleted by `std::constant_wrapper` (P2781R8)
 - in LWG 10:30 AM today
- "non-type template parameter" term was removed by P2841R1 (C++26)

Conclusion

History is messy. `std::nontype` may be a bad feature now, or at least have a terrible name.

std::function_ref refresher

`std::function_ref` is a "function wrapper" similar to `std::function`, `std::move_only_function`, `std::copyable_function`, but without ownership.

`std::function_ref<R(Args...) cv noex>` stores:

- *bound-entity* — *BoundEntityType* (basically a `cv void*`)
- *thunk_ptr* — `R(*) (BoundEntityType, Args...) noex`



NOTE: For illustration purposes, we'll ignore `cv` and `noex` for now.

std::nontype in std::function_ref

```
template<class F>          function_ref(F*);  
template<class F>          function_ref(F&&);  
template<auto f>           function_ref(nontype_t<f>);  
template<auto f, class U>  function_ref(nontype_t<f>, U&&);  
template<auto f, class T>  function_ref(nontype_t<f>, T*);
```

```
template<auto f>  
function_ref(nontype_t<f>) {  
    this->bound-entity = {};  
    this->thunk_ptr = [] (BoundEntityType, Args&&... args) -> R {  
        return f(std::forward<Args&&>(args)...);  
    };  
}
```

Proposed alternative: `std::stateless`

```
template<class F>          function_ref(F*);
template<class F>          function_ref(F&&);
template<class F>          function_ref(stateless_t, const F&);
template<class F, class U> function_ref(stateless_t, const F&, U&&);
template<class F, class T> function_ref(stateless_t, const F&, T*);
```

```
template<class F, class T>
function_ref(stateless_t, const F&) {
    this->bound-entity = {};
    this->thunk_ptr = [] (BoundEntityType, Args&&... args) -> R {
        return F{}(std::forward<Args&&>(args)...);
    };
}
```

Alternatives considered

- remove `std::nontype`, revisit in C++29
 - ... but this feature is too important
- wait for P1045R1 `constexpr` function parameters
 - ... but paper looks abandoned, no C++29 guarantee
- expose the *BoundEntityType*
 - ... but leaking non-portable implementation details
- rename `std::nontype`
 - ... to what? confusion with `std::constant_wrapper`

Why not just use `std::constant_wrapper`

`std::constant_wrapper<int(*)()>` already behaves *much* like the function pointer it wraps. Therefore:

- `F&&` and `F*` constructors are candidates, but lose in overload resolution
- `std::constant_wrapper` also works in `std::move_only_function`
 - ... but with totally different meaning?!
- `std::nontype` reference impl. also supports `std::move_only_function`
 - ... which would make a lot more sense
- design still ages poorly given `constexpr` function parameters

Most viable solutions

- use `std::stateless` constructor tag instead
- use `std::constant_wrapper` instead of `std::nontype`
- rename `std::nontype`
 - **Author's proposal:** `std::constant_arg` / `std::constant_arg_t`